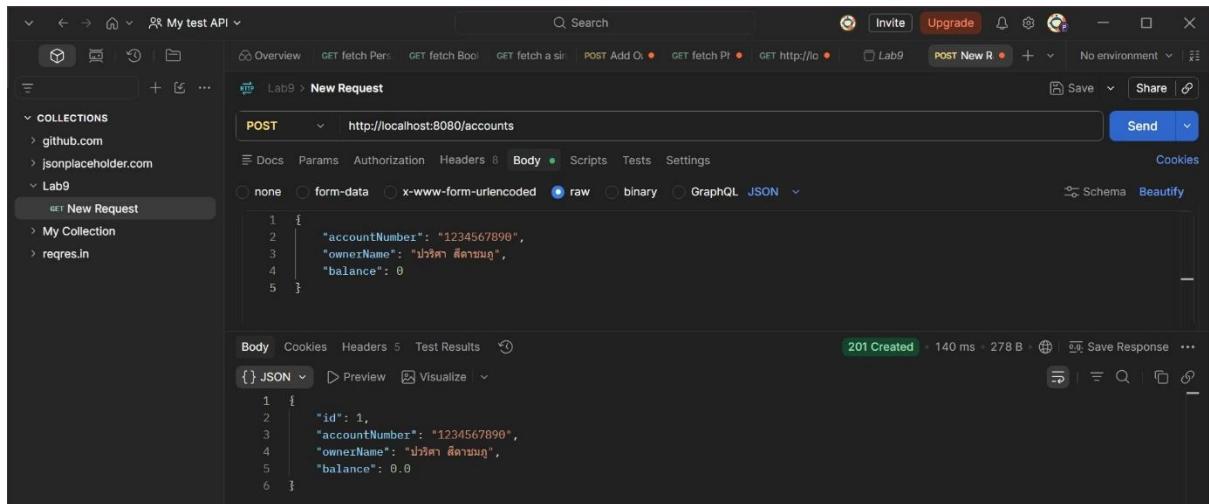


Lab9 Spring Boot Transaction

1. การสร้าง Account

รูปภาพ :



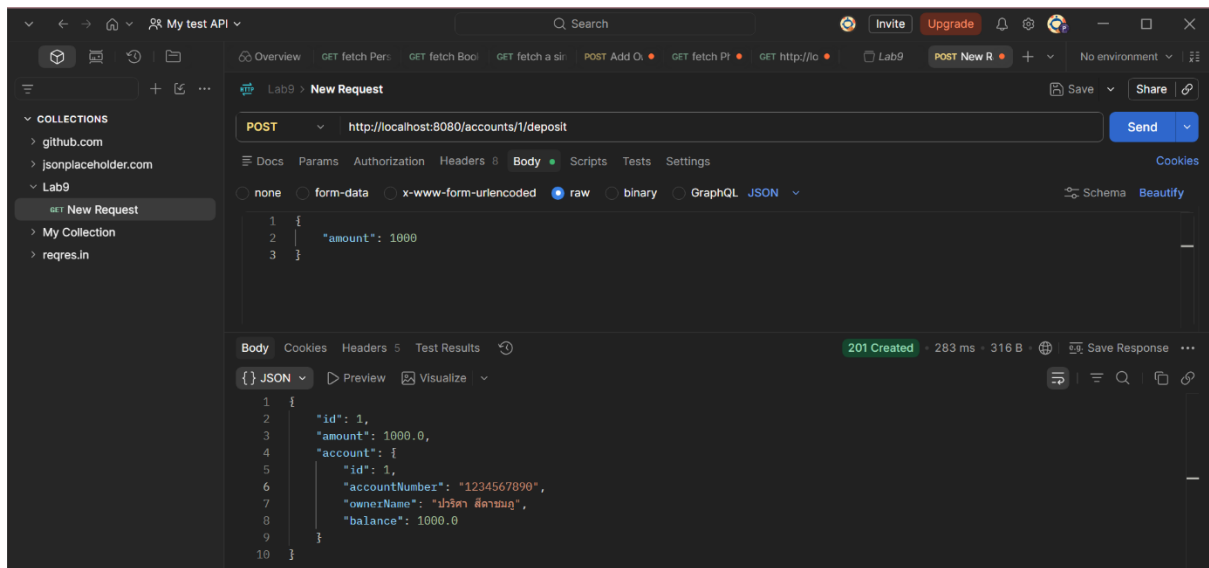
อธิบายผล :ส่ง POST `http://localhost:8080/accounts` พร้อม body

```
{
  "accountNumber": "1234567890",
  "ownerName": "ปวริศา สีดาชมภู",
  "balance": 0
}
```

ได้ผลลัพธ์ 201 Created พร้อม id: 1 และ balance: 0.0 สร้าง Account ใหม่สำเร็จ ถูก insert เข้า database และได้ id กลับมาอัตโนมัติจาก GenerationType.IDENTITY

2. การฝากเงินสำเร็จ

รูปภาพ :

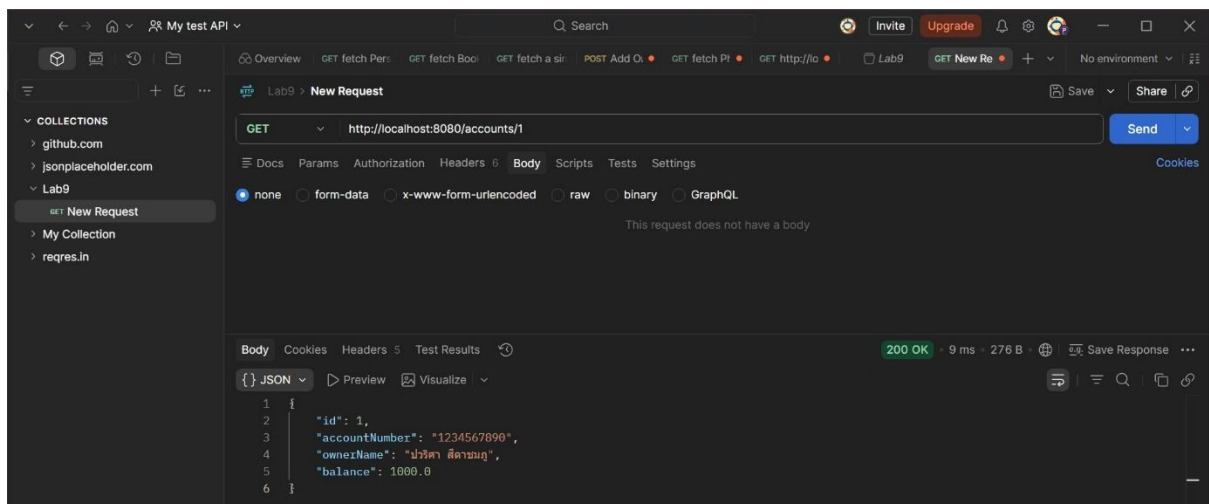


อธิบายผล :ส่ง POST `http://localhost:8080/accounts/1/deposit` พร้อม body `{\"amount\": 1000}`

ได้ผลลัพธ์ 201 Created response กลับมาเป็น DepositTransaction object ที่มี id: 1, amount: 1000.0 และ nested account ที่ balance อัปเดตเป็น 1000.0 แล้ว

3. ตรวจสอบ Account

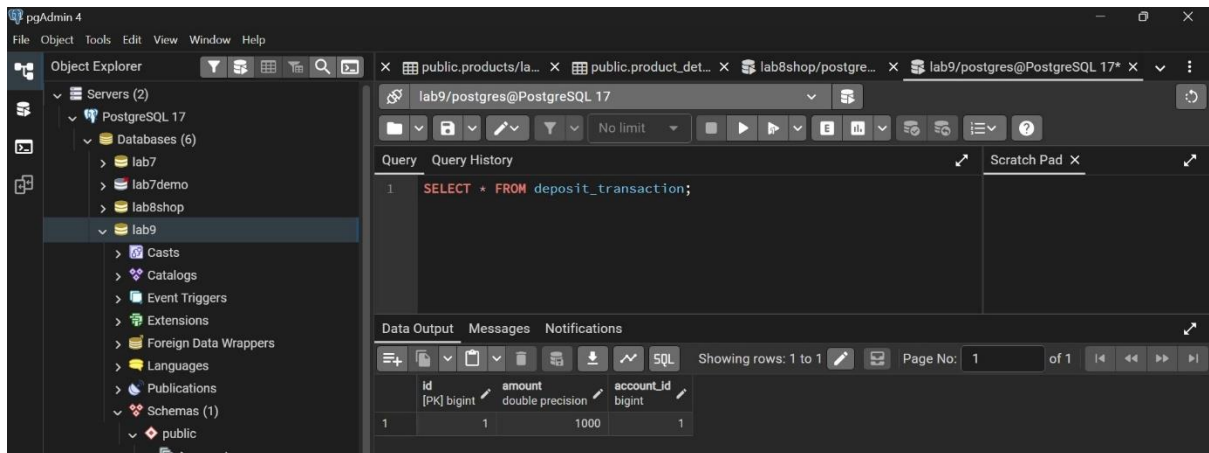
รูปภาพ :



อธิบายผล :ส่ง GET `http://localhost:8080/accounts/1` ได้ 200 OK พร้อม balance: 1000.0 การฝากเงินในรูปแบบที่ 2 ได้บันทึก balance ลง database จริง

4. ตรวจสอบ DepositTransaction

รูปภาพ :

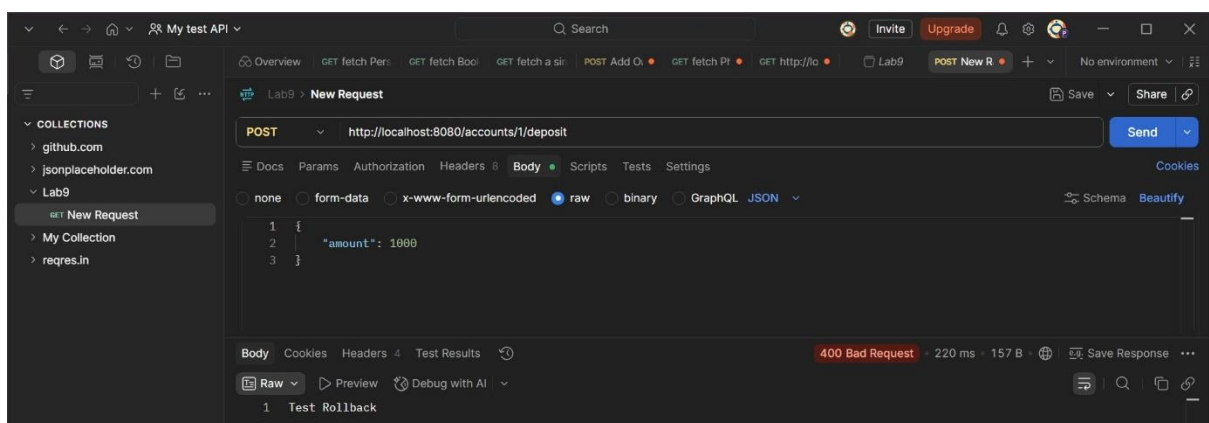


อธิบายผล :รัน `SELECT * FROM deposit_transaction;` พบ 1 แถว (`id=1`, `amount=1000`, `account_id=1`) record การฝากถูกเก็บลงตาราง `deposit_transaction` จริง และเชื่อมกับ `account_id=1` ถูกต้องตามความสัมพันธ์ Many-to-One

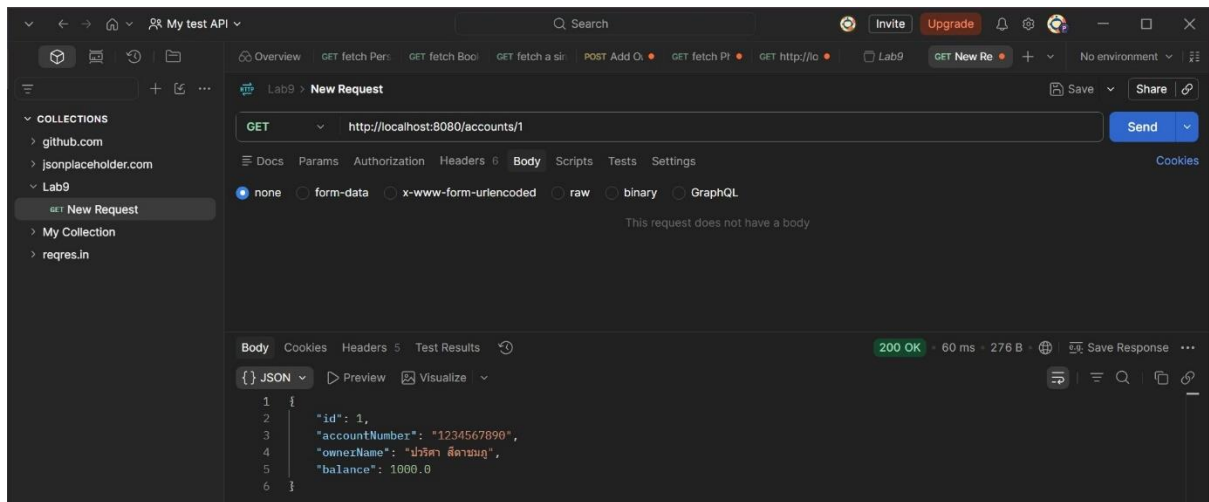
5. ทดลอง Rollback

รูปภาพ :

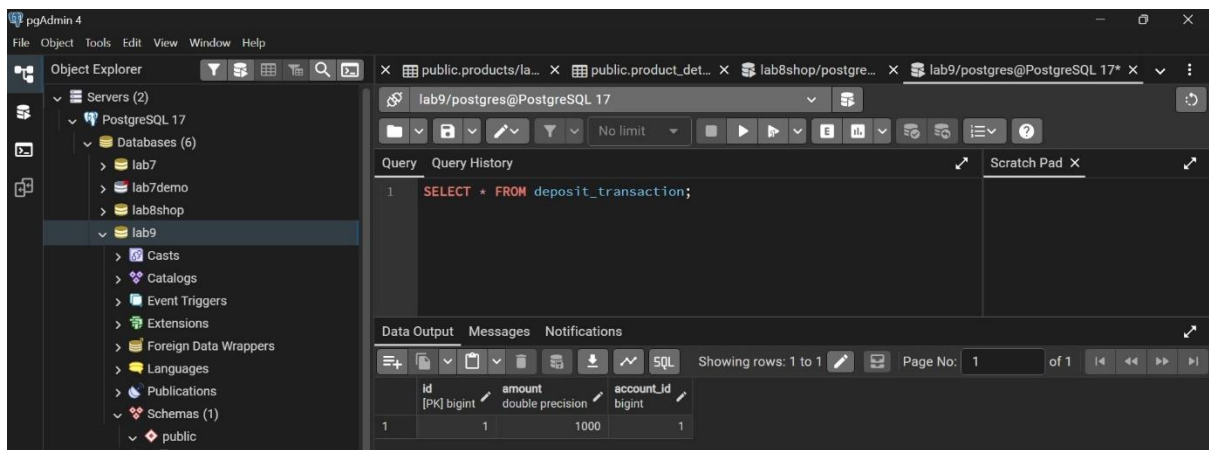
ยิง Request ขึ้น



ตรวจสอบว่าข้อมูลใน Postman



ตรวจสอบว่าข้อมูลใน pgAdmin

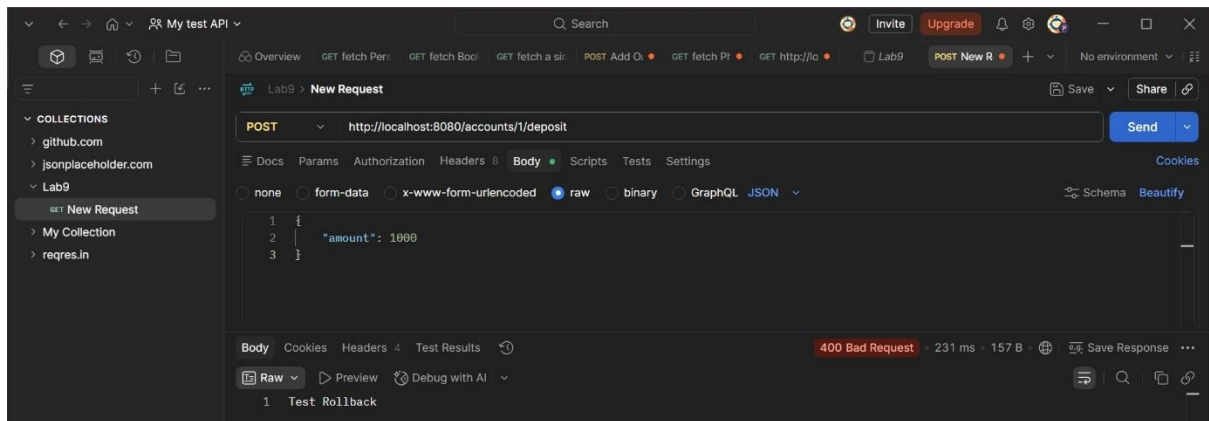


อธิบายผล :ส่ง deposit อีกครั้ง (amount: 1000) แต่ครั้งนี้ได้ถูกแก้ไขให้ throw RuntimeException("Test Rollback") ก่อนจบ method ได้ 400 Bad Request พร้อมข้อความ "Test Rollback" เช็ค GET /accounts/1 balance ยังเป็น 1000.0 เหมือนเดิม เช็ค pgAdmin ตาราง deposit_transaction ยังมีแค่ 1 แถวเหมือนเดิม

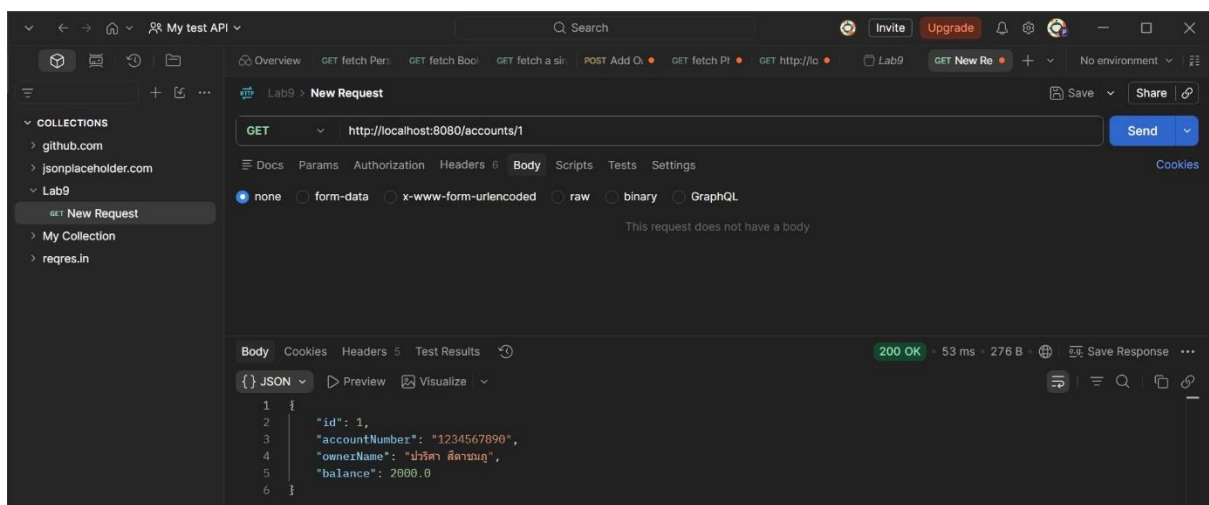
6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional

รูปภาพ : มี @Transactional เหมือนข้อ5

ไม่มี @Transactional ยิง Request ซ้ำ

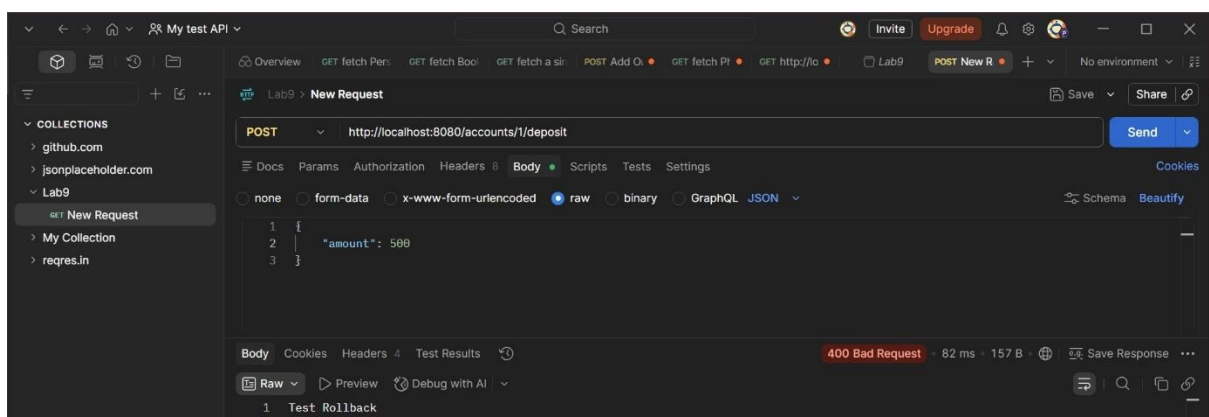


ไม่มี @Transactional ตรวจสอบว่าข้อมูลใน Postman

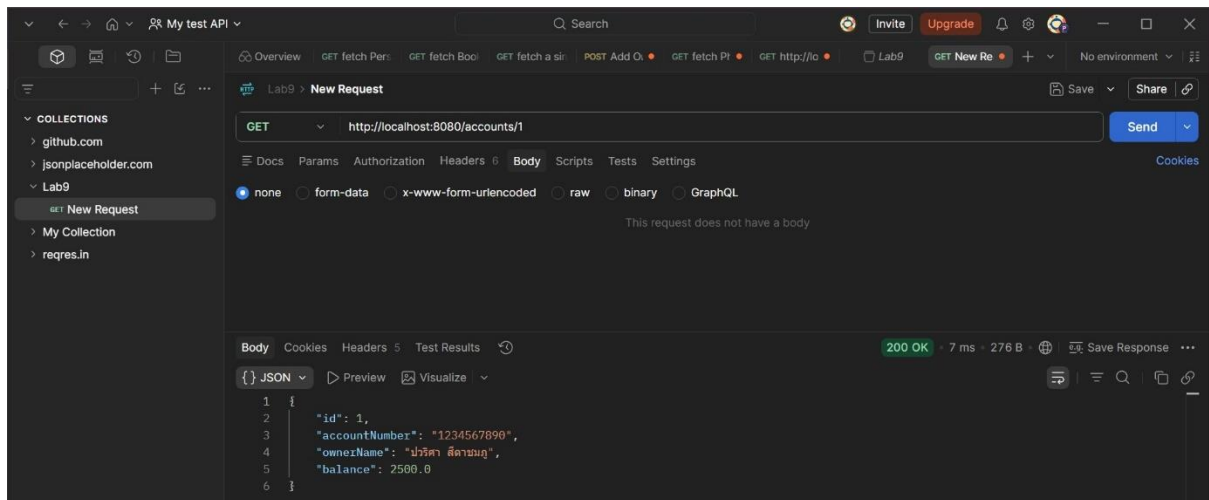


ไม่มี @Transactional ตรวจสอบว่าข้อมูลใน pgAdmin มันไม่บันทึกลง database เลยยิงใหม่อีกรอบ

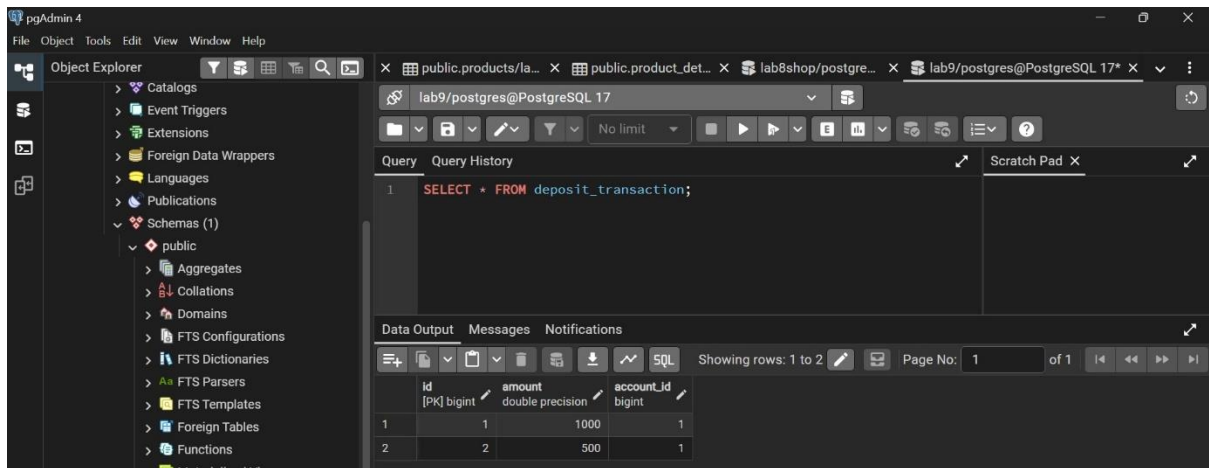
ไม่มี @Transactional ยิง Request ซ้ำรอบ 2



ไม่มี @Transactional ตรวจสอบว่าข้อมูลใน Postman รอบ 2



ไม่มี @Transactional ตรวจสอบว่าข้อมูลใน pgAdmin รอบ 2



อธิบายผล :ลบ @Transactional ออกจาก method แล้วส่ง deposit (amount: 1000) เข้าได้ 400 Bad Request เหมือนเดิม เช็ค GET /accounts/1 balance กลายเป็น 2000.0 เช็ค pgAdmin ยังไม่มีแถวใหม่ จึงลองยิงซ้ำ

ส่ง deposit อีกครั้งด้วย amount: 500 balance เพิ่มขึ้น 2500.0 เช็ค pgAdmin ตาราง deposit_transaction พบ 2 แถว (id=1, amount=1000 จากการฝากสำเร็จครั้งแรก และ id=2, amount=500 จารอบ 2)

7. คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

ตอบ @Transactional เป็น annotation ที่บอกให้ method ทำงานภายใต้ database transaction เดียว ทุกคำสั่งที่เกี่ยวข้องกับ database ภายใน method จะถูกมองเป็นหน่วยเดียวกัน ถ้าทุกคำสั่งสำเร็จหมด ระบบจะ COMMIT แต่ถ้ามีคำสั่งใดคำสั่งหนึ่งเกิด error ระหว่างทาง ระบบจะ

ROLLBACK ยกเลิกการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นใน method นั้น กลับไปเป็นสถานะก่อนเริ่ม transaction

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

ตอบ เพราะถ้าไม่ใช้ transaction แล้วเกิด error ระหว่าง อัปเดต balance ของ Account หรือ สร้าง record ใหม่ใน DepositTransaction เพื่อเก็บ history จะทำให้ข้อมูลไม่ตรงกัน balance เพิ่มขึ้นแล้วแต่ไม่มีหลักฐานการฝากเก็บไว้ การใช้ @Transactional ทั้งสอง step จะสำเร็จไปด้วยกัน หรือไม่สำเร็จไปด้วยกันเลยทีเดียว

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

ตอบ COMMIT คือการยืนยันว่าการเปลี่ยนแปลงข้อมูลทั้งหมดใน transaction นั้นถูกต้องและสมบูรณ์ ให้บันทึกลง database อย่างถาวร ข้อมูลจะเปลี่ยนจริง

ROLLBACK คือการยกเลิกการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นใน transaction นั้น กลับไปเป็นสถานะก่อนที่ transaction จะเริ่มต้น เหมือนกับว่าไม่มีอะไรเกิดขึ้นเลย

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ตอบ เป็นความสัมพันธ์แบบ One-to-Many

1 Account สามารถมี DepositTransaction ได้หลายรายการ (ฝากเงินได้หลายครั้ง)

1 DepositTransaction อ้างอิงไปที่ Account เดียว เท่านั้น (ฝากเข้าบัญชีใดบัญชีหนึ่ง)

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ตอบ ข้อมูลจะไม่เปลี่ยนแปลง Spring จะ ROLLBACK การเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นภายใน method นั้นโดยอัตโนมัติ กลับไปเป็นสถานะเดิมก่อนเริ่ม transaction

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ตอบ balance เพิ่มขึ้นทุกครั้งที่ยิง ทั้งที่ response กลับมาเป็น error ทุกครั้ง